

ITAI-1378-Comp Vision-Artificial Intel

Iman Haamid

Lab 04 - Chihuahua or Muffin Workshop

This project focused on developing a neural network classifier designed to distinguish between a **MUFFIN** and a **CHIHUAHUA**. The funny thing is, some of the chihuahuas looked really like muffins.

First I needed to understand the concept of neural network classifiers that they used in the workshop. A neural network classifier, a machine learning model, is designed to categorize data. Its structure, inspired by the human brain, uses interconnected layers of "neurons" or nodes, allowing it to learn how to sort information into distinct categories. Overfitting was my biggest challenge. My model initially performed well on training data but poorly on validation, highlighting the need for generalization. Tackling this, I used techniques like dropout and early stopping. This iterative refinement process taught me machine learning is about continuous improvement, not first-attempt perfection.

Real-World Application:

Product Recommendation: Online stores classify users' past purchases and browsing behavior to recommend new products they are likely to buy. The products with the most similar feature vectors are then recommended to the user. This approach allows the system to recommend new products that are visually similar to what the user is currently viewing, even if the products don't share the same keywords or categories. For example, if a user is looking at a blue striped shirt, the system could recommend other blue striped shirts, or even a similar-looking dress or a different type of striped apparel.

Concepts or steps the workshop used were building out the neural network, loading the data, training the model on the data and then visualizing the results.

1. Setup & Data Loading

Import essential libraries (TensorFlow, PyTorch, NumPy, Matplotlib). Load and split the dataset into training and validation/testing sets.

2. Model Definition

Define the neural network architecture by creating layers (input, hidden, output), choosing activation functions (ReLU, sigmoid, softmax), and compiling the model with an optimizer (Adam, SGD) and loss function (categorical cross-entropy).

3. Training the Model

Train the network over epochs, feeding data in batches. The model makes predictions, calculates error via the loss function, and the optimizer adjusts weights through backpropagation to minimize loss and improve accuracy.

4. Evaluation & Visualization

Evaluate model performance using an unseen test set, typically measuring accuracy. Visualize results by plotting training/validation accuracy, displaying a confusion matrix, and showing example predictions to check for overfitting and identify areas of struggle.

Improving a neural network involves iterative experimentation. Key techniques include:

- **Acquire More Data:** Boost performance by collecting more data for better generalization.
- **Data Augmentation:** Generate new training examples from existing data (e.g., image rotations) to enhance diversity and prevent overfitting when new data isn't available.
- **Feature Engineering:** Create or transform features (e.g., derive "day of the week" from a date) to make data more meaningful.

References

Amazon Web Services (AWS). (n.d.). *What is Overfitting?* Retrieved from <https://aws.amazon.com/what-is/overfitting/>

DigitalOcean. (n.d.). *Dropout Regularization in Deep Learning*. Retrieved from <https://www.digitalocean.com/community/tutorials/dropout-regularization-deep-learning>

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1), 1929-1958.